Experiment 10 – Mini Project

TokenSense: AI Token Utilization Prediction by Profession

Algorithms used: Random Forest Regression (Module 3) and K-Means Clustering (Module 4)

Name	Roll No.	Class / Division	Date
		TE-B	

---

1. Aim

To build a web application that predicts how many AI tokens a person will need per day and per month, split into **input tokens** (what the person feeds to the AI) and **output tokens** (what the AI generates), based on their profession and usage habits, and to group users into usage types.

2. Problem Statement

AI assistants such as ChatGPT, Claude, Gemini and GitHub Copilot are now part of daily work for developers, students, writers, teachers, doctors and lawyers. Every one of these services measures and bills usage in **tokens** (about 0.75 words per token). Users cannot easily answer simple questions such as “How many tokens will I use this month?” or “Which model tier can I afford?”. Token usage depends on the profession (a developer pastes code and error logs, a content writer asks for long replies), on habits (number of chats, messages per chat, prompt length, uploaded files, AI agents) and on the calendar (weekends, deadline or exam weeks). TokenSense learns these patterns from data and gives each person a personal token budget and cost estimate.

3. Dataset

Real per-user token logs are private, so a realistic **synthetic dataset** was generated in Python (src/generate_data.py, random seed 42). The rules follow how large language model billing actually works: every new message re-sends the whole conversation, so input tokens grow roughly with the square of the messages per session; each uploaded document adds about 2,500 tokens; AI agents multiply the context about 2.5 times; weekends, Fridays and deadline weeks change the number of sessions; and ±12% random noise is added to every value.

- **File:** token_usage_daily.csv with **9,000 rows** (300 users × 30 days) covering 12 professions.
- **Input features (11):** profession, day of week, model tier, experience in years, deadline/exam week, uses AI agents, chat sessions per day, messages per session, average prompt length in words, documents uploaded, coding task.
- **Targets (2):** input_tokens and output_tokens per day.
- **Split:** 7,200 rows for training and 1,800 unseen rows for testing (80/20).

4. Algorithms Used

4.1 Random Forest Regression (Module 3)

A Random Forest is an ensemble of many decision trees. Each tree is trained on a bootstrap sample (rows drawn at random with replacement), and at every split it considers only a random subset of the features. A regression tree keeps splitting the data so that the squared error inside each branch becomes as small as possible, and it predicts the average target value of the rows in a leaf. The forest predicts the **average of all its trees**. Averaging many different trees removes most of the overfitting of a single tree while keeping its ability to learn non-linear relationships.

This is important for token prediction because usage is strongly non-linear: doubling the messages in one chat roughly quadruples the input tokens, and switching on AI agents multiplies everything. In this project one multi-output forest with **200 trees** predicts both input and output tokens. Categorical features (profession, day, model tier) are one-hot encoded with a ColumnTransformer inside a scikit-learn Pipeline, so exactly the same preprocessing runs during training and inside the web application. On the unseen test rows the model reached **R^2 = 0.915**. Feature importance shows that the biggest drivers of token usage are chat sessions per day (35%), messages per session (25%), use of AI agents (18%), prompt length (17%).

Target	MAE (tokens)	RMSE (tokens)	R^2
input_tokens	11,298	26,145	0.924
output_tokens	2,116	3,557	0.906
overall	6,707	14,851	0.915

4.2 K-Means Clustering (Module 4)

K-Means is an unsupervised learning algorithm that divides data into k groups (clusters) without any labels. It works in four steps:

1. Choose k starting centroids.
2. Assign every point to its nearest centroid using Euclidean distance.
3. Move each centroid to the mean of the points assigned to it.
4. Repeat steps 2 and 3 until the assignments stop changing.

Each of the 300 users is summarised by four features calculated from their 30 days of usage: log of average daily tokens, burstiness (standard deviation divided by mean), output share of tokens, and weekend-to-weekday usage ratio. The features are standardised first so that no feature dominates the distance. The number of clusters **k = 4** was chosen with the elbow method (inertia) and the silhouette score (0.25). The clusters were named from their centroids:

User type	Users	Avg tokens / day	Most common professions
Light / Irregular User	53	36,912	Digital Marketer, Designer, Business Analyst
Steady 9-to-5 Professional	98	44,607	Customer Support, Doctor, Teacher
Anytime Learner / Creator	57	88,156	Student, Researcher, Content Writer
Power User (Heavy Context)	92	1,53,347	Data Scientist, Software Developer, Lawyer

How the two algorithms work together: when a new user submits the form, the Random Forest predicts their tokens for all seven days of a week. The same four profile features are calculated from that forecast, and the trained K-Means model assigns the user to one of the four types.

5. System Design: Frontend, Backend and Their Connection

5.1 Frontend

The frontend is a single-page web interface written in plain HTML, CSS and JavaScript (templates/index.html, static/style.css, static/app.js), without any framework. It has four tabs:

- **Predict:** a form for the user's profile, then result cards for input tokens, output tokens, today's cost and monthly requirement, the K-Means user type, the monthly cost on small, medium and large models, and a 7-day forecast chart for a normal week and a deadline week.
- **Model Insights:** Random Forest test metrics and feature importance, the four K-Means clusters, and all training charts.
- **Training Data:** a sample of the training CSV and summary statistics.

- **About:** the algorithm table and the architecture diagram.

Choosing a profession automatically fills in that profession's typical habits taken from the dataset, so a user can get a prediction with one click and then adjust the numbers.

5.2 Backend

The backend is a Python **Flask** server (app.py). When it starts, it loads the trained Random Forest pipeline and the K-Means model (saved with joblib by src/train.py) and exposes a small REST API that returns JSON:

- **POST /api/predict:** Random Forest prediction, 7-day and monthly forecast, cost in USD and INR, and the K-Means user type.
- **GET /api/insights:** test metrics, feature importance and cluster details from training.
- **GET /api/meta:** dropdown options and the default habits for each profession.
- **GET /api/dataset:** a preview of the training data.

5.3 Connection Between Frontend and Backend

When the user clicks "Predict my token usage", app.js reads the form and sends it to /api/predict as JSON using the browser's fetch() function. Flask converts the JSON into a one-row pandas DataFrame and passes it through the same pipeline used in training (one-hot encoding, then the Random Forest). It then runs K-Means on the weekly forecast, calculates the cost, and sends everything back as one JSON response. JavaScript renders the response into cards, tables and charts without reloading the page. Model training (src/train.py) runs separately and only writes the model files that the server reads, so the machine learning code and the web code stay cleanly separated.

Layer	Technology	Main files
Frontend	HTML, CSS, JavaScript (fetch API)	templates/index.html, static/app.js, static/style.css
Backend	Python, Flask REST API	app.py
Machine learning	scikit-learn, pandas, NumPy	src/train.py, src/profiles.py
Saved models	joblib, JSON	models/rf_token_model.joblib, models/kmeans_users.joblib, models/insights.json
Data	CSV (synthetic)	data/token_usage_daily.csv (generated by src/generate_data.py)

6. Result

TokenSense predicts daily input and output tokens with **R^2 = 0.915** and an average error of 6,707 tokens on unseen data. For example, a software developer with 8 chats a day on a medium-tier model is predicted to feed about 80,700 tokens to the AI and receive about 18,700 tokens back (about ₹44 per day, 2.79 million tokens per month), and is classified as a "Power User (Heavy Context)".

7. Conclusion

Random Forest regression learned the non-linear way AI token usage grows with chats, messages, prompt length and agents, and predicted it accurately for every profession. K-Means found four meaningful user types without any labels, which helps match each person to the right token budget or plan. Together they give a practical tool for planning AI usage and cost. **Future scope:** train on real exported usage logs, add time-series forecasting, and connect directly to AI providers' usage APIs.

8. Screenshots

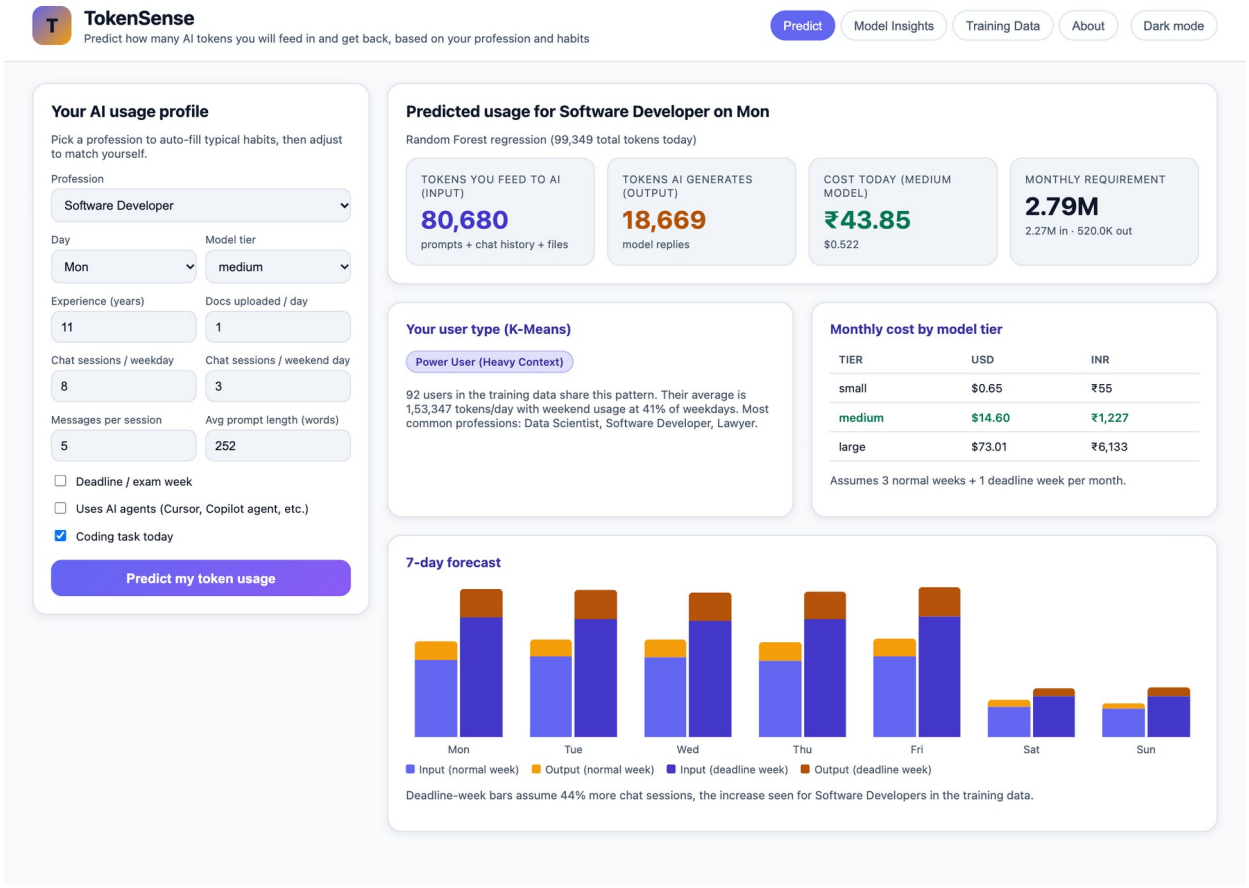


Figure 1: Predict tab – Software Developer: input/output tokens, cost, user type and 7-day forecast

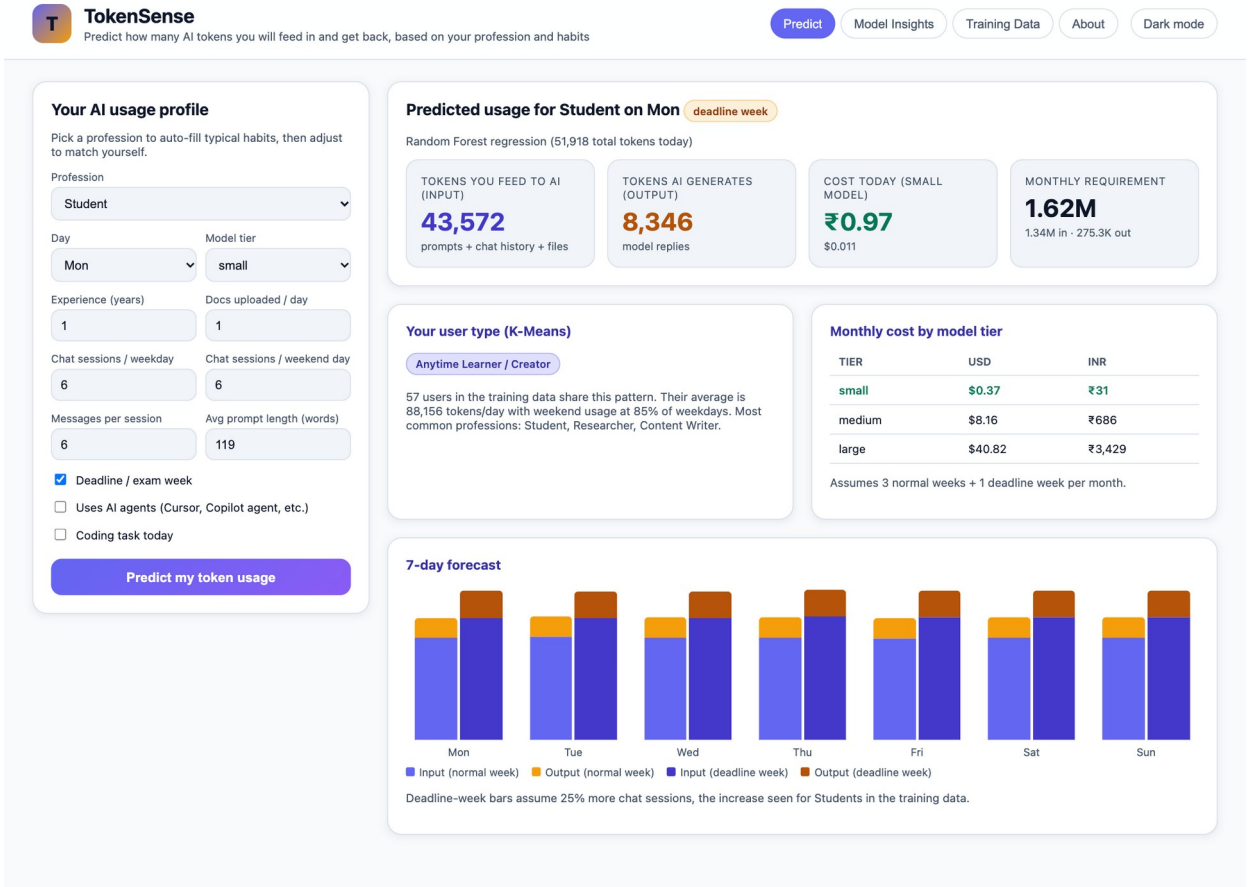


Figure 2: Predict tab – Student in exam week

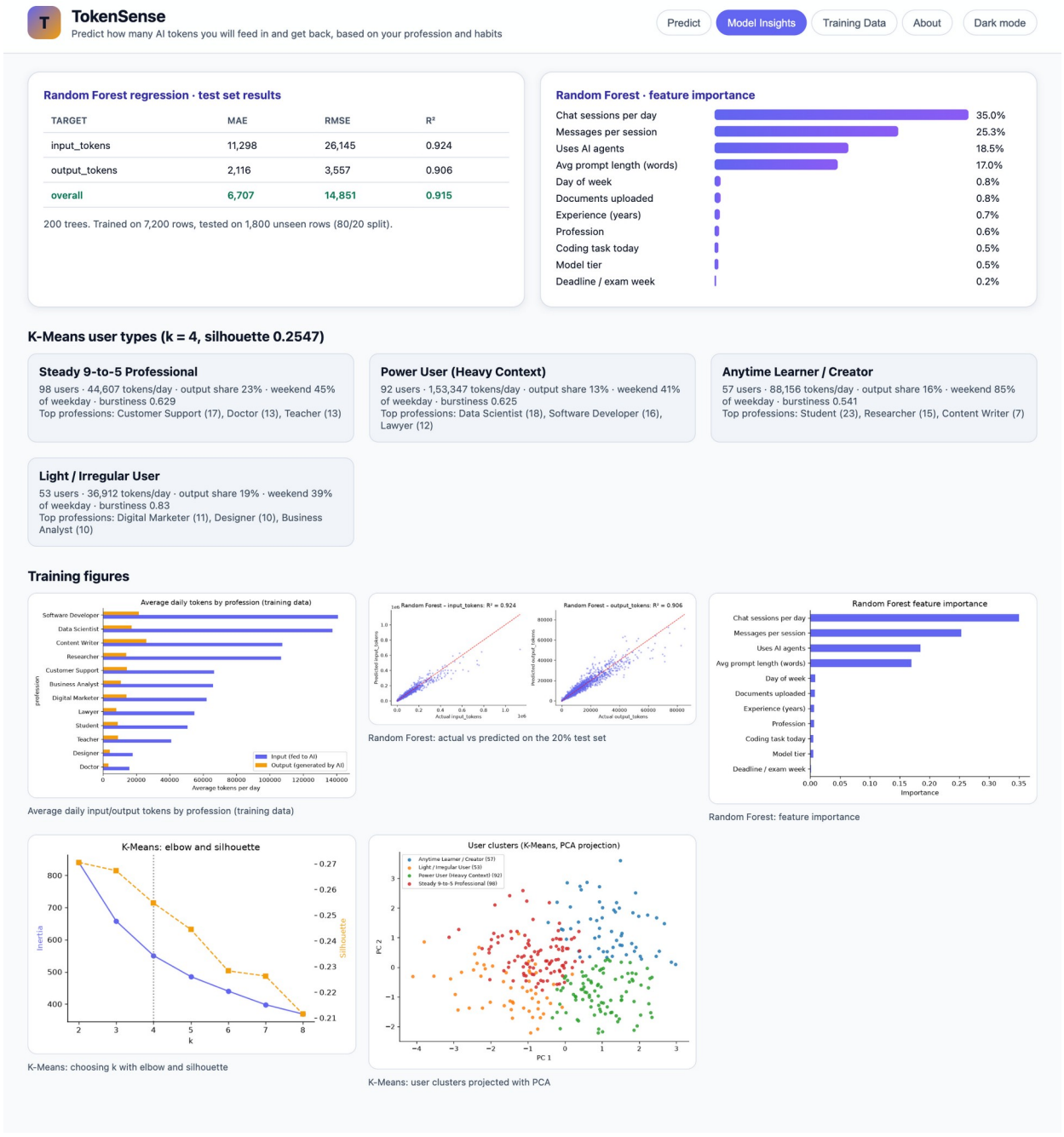


Figure 3: Model Insights tab – Random Forest metrics, feature importance and K-Means clusters

TokenSense
Predict how many AI tokens you will feed in and get back, based on your profession and habits

Predict | Model Insights | Training Data | About | Dark mode

token_usage_daily.csv · 9,000 rows (300 users × 30 days), random sample of 25

Used by Random Forest and K-Means. Targets: **input_tokens** and **output_tokens**.

USER_ID	PROFESSION	EXPERIENCE_YEARS	MODEL_TIER	USES_AGENTS	DAY_OF_WEEK	DEADLINE_WEEK	SESSIONS_PER_DAY	TURNS_PER_SESSION	AVG_PROMPT_WORDS	DOCS_UPLOA
U175	Doctor	21	medium	0	Thu	0	0	4	80	1
U100	Content Writer	19	small	0	Tue	0	9	7	178	0
U193	Software Developer	21	medium	1	Wed	0	9	6	217	2
U287	Customer Support	5	small	1	Sat	0	1	5	91	0
U045	Researcher	18	large	0	Wed	0	2	5	220	5
U075	Student	2	medium	0	Mon	0	8	6	163	1
U215	Customer Support	14	small	0	Mon	0	12	4	65	0
U058	Designer	14	small	0	Mon	1	4	4	92	2
U179	Customer Support	5	small	0	Wed	1	11	5	90	1
U279	Student	1	large	0	Sat	0	5	5	130	0
U003	Student	1	large	0	Tue	0	4	5	133	1
U097	Software Developer	12	medium	1	Tue	0	9	2	343	0

Target statistics

	INPUT_TOKENS	OUTPUT_TOKENS	TOTAL_TOKENS
count	9,000	9,000	9,000
mean	72,254	12,615	84,869
std	92,709	12,037	1,02,987
min	0	0	0
25%	17,920	4,357	22,795
50%	41,010	9,006	50,472
75%	88,340	17,133	1,06,123
max	12,73,524	1,40,143	13,50,295

Figure 4: Training Data tab – dataset sample and statistics

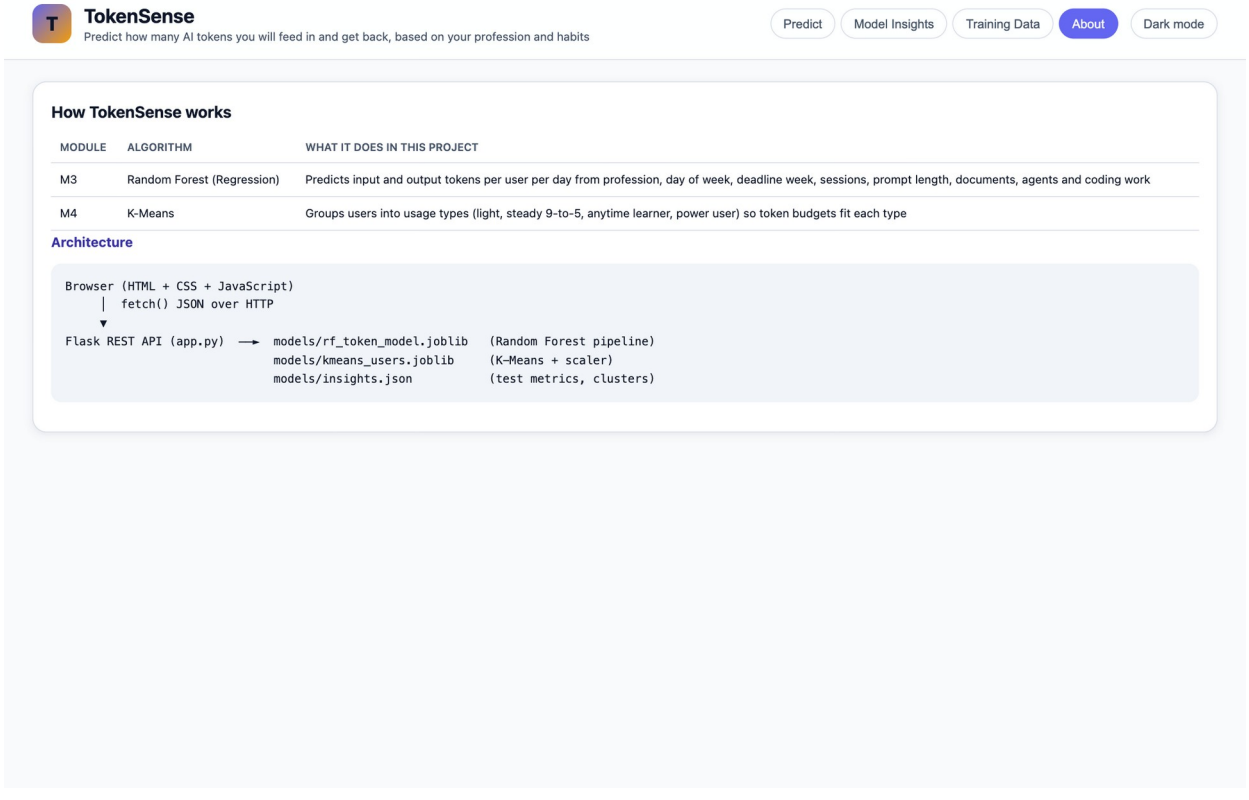


Figure 5: About tab – algorithms and architecture

9. Training Graphs

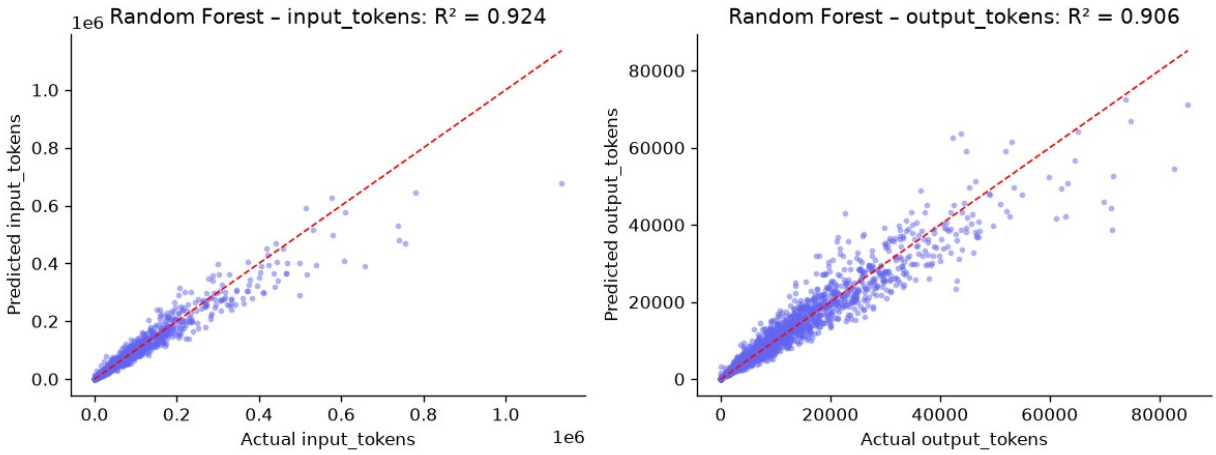


Figure 6: Random Forest – actual vs predicted tokens on the test set

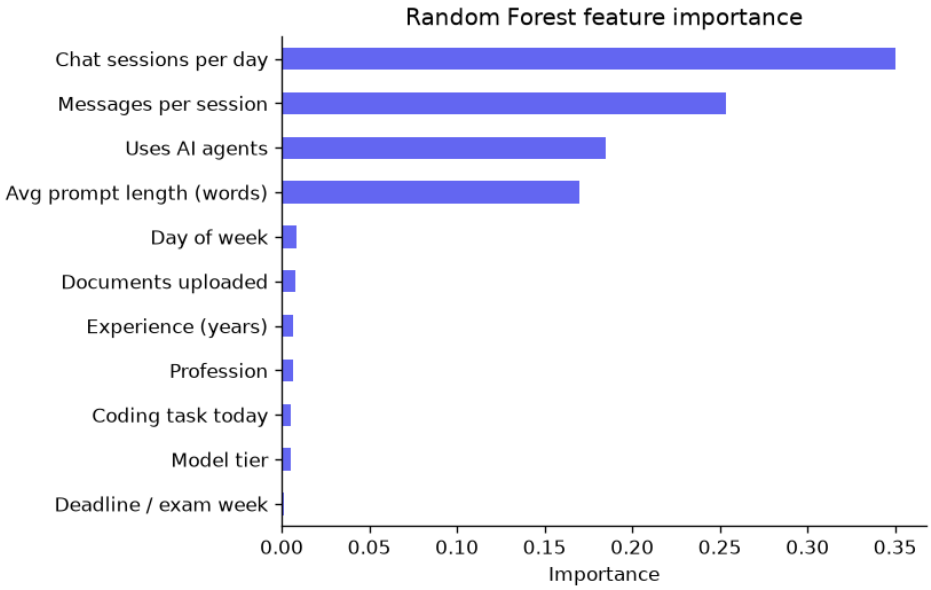


Figure 7: Random Forest – feature importance

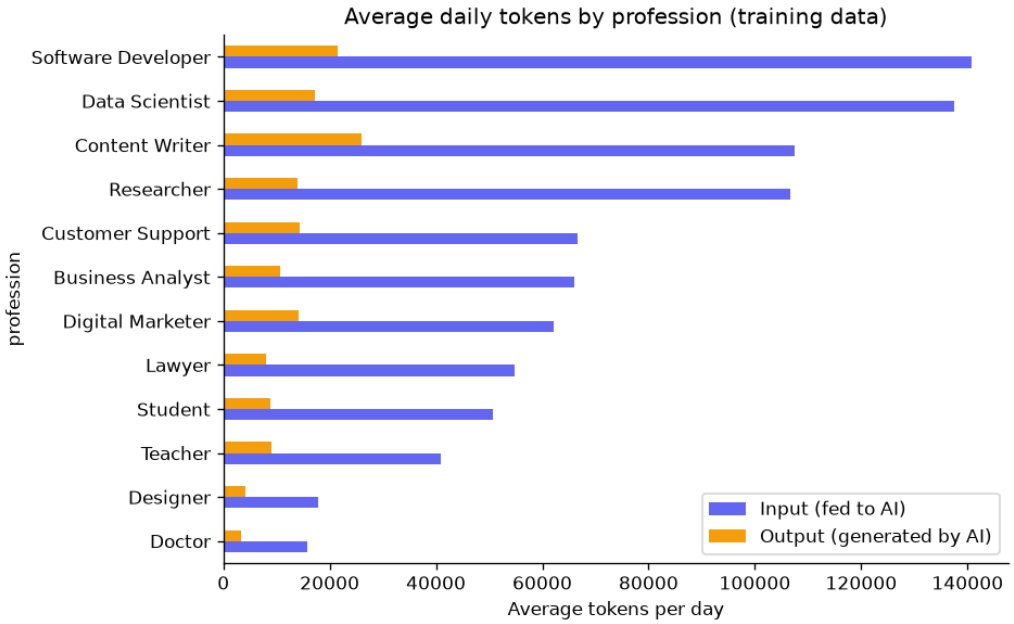


Figure 8: Average daily tokens by profession (training data)

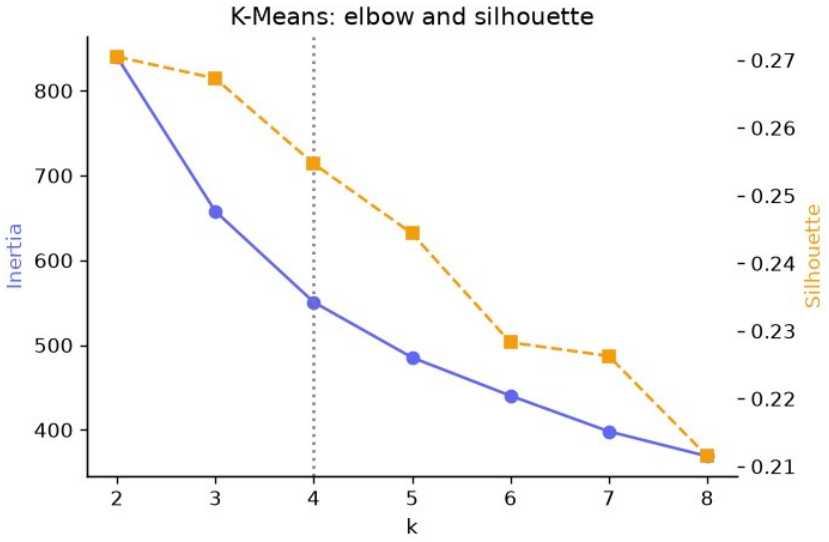


Figure 9: K-Means – elbow method and silhouette score for choosing k

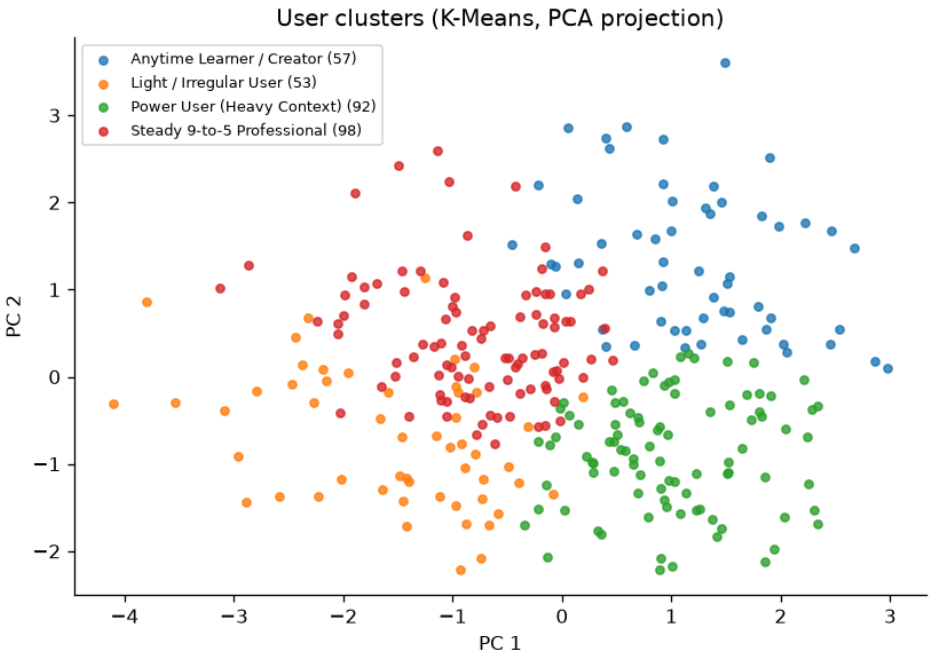


Figure 10: K-Means – user clusters (PCA projection)

10. Important Code

10.1 Random Forest regression training (src/train.py)

```
def train_random_forest(df):
    X, y = df[FEATURES], df[TARGETS]
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=SEED)

    pipe = Pipeline([
        ("prep", ColumnTransformer([
            ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), CATEGORICAL),
            ("num", "passthrough", NUMERIC),
        ])),
        ("model", RandomForestRegressor(n_estimators=N_ESTIMATORS, min_samples_leaf=2,
                                        random_state=SEED, n_jobs=-1)),
    ])
    pipe.fit(X_train, y_train)
    pred = pipe.predict(X_test)

    metrics = {
        t: {"mae": round(mean_absolute_error(y_test[t], pred[:, i]), 1),
            "rmse": round(root_mean_squared_error(y_test[t], pred[:, i]), 1),
            "r2": round(r2_score(y_test[t], pred[:, i]), 4)}
        for i, t in enumerate(TARGETS)
    }
    metrics["overall"] = {"mae": round(mean_absolute_error(y_test, pred), 1),
                          "rmse": round(root_mean_squared_error(y_test, pred), 1),
                          "r2": round(r2_score(y_test, pred), 4)}
    for k, m in metrics.items():
        print(f"[RF] {k:14s} MAE={m['mae']:>9}  RMSE={m['rmse']:>9}  R2={m['r2']}")

    # Feature importance, with one-hot columns summed back into their original feature
    names = pipe.named_steps["prep"].get_feature_names_out()
    importance = {}
    for name, imp in zip(names, pipe.named_steps["model"].feature_importances_):
        col = name.split("__", 1)[1]
        source = next((c for c in CATEGORICAL if col.startswith(c + "_")), col)
        importance[source] = importance.get(source, 0) + float(imp)
    importance = dict(sorted(((k, round(v, 4)) for k, v in importance.items()), key=lambda kv: -kv[1]))

    return pipe, metrics, importance, (y_test, pred), (len(X_train), len(X_test))
```

10.2 K-Means clustering (src/train.py)

```
def train_kmeans(df):
    rows = []
    for uid, g in df.groupby("user_id"):
        prof = build_profile(g["day_of_week"], g["input_tokens"], g["output_tokens"])
        prof.update(user_id=uid, profession=g["profession"].iloc[0],
                    avg_daily_tokens=float((g["input_tokens"] + g["output_tokens"]).mean()))
        rows.append(prof)
    profiles = pd.DataFrame(rows)
    scaler = StandardScaler()
    Z = scaler.fit_transform(profiles[PROFILE_FEATURES])

    elbow = []
    for k in range(2, 9):
        km = KMeans(n_clusters=k, n_init=10, random_state=SEED).fit(Z)
        elbow.append({"k": k, "inertia": round(km.inertia_, 2),
                      "silhouette": round(silhouette_score(Z, km.labels_), 4)})

    km = KMeans(n_clusters=N_CLUSTERS, n_init=10, random_state=SEED).fit(Z)
    profiles["cluster"] = km.labels_
    centers = pd.DataFrame(scaler.inverse_transform(km.cluster_centers_), columns=PROFILE_FEATURES)
    names = name_clusters(centers)
    profiles["cluster_name"] = profiles["cluster"].map(names)
    silhouette = round(silhouette_score(Z, km.labels_), 4)
    ...
```

10.3 Backend prediction API (app.py)

```
@app.post("/api/predict")
def predict():
    body = request.get_json(force=True)
    try:
        row = parse_inputs(body)
    except (KeyError, ValueError) as e:
        return jsonify({"error": str(e)}), 400
    weekend_sessions = float(body.get("weekend_sessions", row["sessions_per_day"]))
```

```
    rf = state["rf"]

    # M3: Random Forest predicts today's input and output tokens
    x = pd.DataFrame([row])[FEATURES]
    inp, out = (max(0.0, v) for v in rf.predict(x)[0])

    normal = rf.predict(week_frame(row, weekend_sessions, 0)).clip(min=0)
    deadline = rf.predict(week_frame(row, weekend_sessions, 1)).clip(min=0)
    weekly = [{"day": d, "input": int(n[0]), "output": int(n[1]),
               "deadline_input": int(dl[0]), "deadline_output": int(dl[1])}
              for d, n, dl in zip(DAYS, normal, deadline)]

    # A typical month = 3 normal weeks + 1 deadline/exam week, scaled to 30 days
    month_in = (3 * normal[:, 0].sum() + deadline[:, 0].sum()) * 30 / 28
    month_out = (3 * normal[:, 1].sum() + deadline[:, 1].sum()) * 30 / 28
    monthly = {
        "input_tokens": int(month_in), "output_tokens": int(month_out),
        "total_tokens": int(month_in + month_out),
        "cost_by_tier": {t: cost(month_in, month_out, t) for t in MODEL_TIERS},
    }

    # M4: K-Means assigns the user type from their predicted week
    km = state["kmeans"]
    profile = build_profile(DAYS, normal[:, 0], normal[:, 1])
    z = km["scaler"].transform(pd.DataFrame([profile])[PROFILE_FEATURES])
    cluster_id = int(km["kmeans"].predict(z)[0])
    cluster = next(c for c in state["insights"]["kmeans"]["clusters"] if c["cluster"] == cluster_id)

    return jsonify({
        "inputs": row,
        "prediction": {"input_tokens": int(inp), "output_tokens": int(out), "total_tokens": int(inp + out),
                       "cost": cost(inp, out, row["model_tier"])},
        "weekly": weekly,
        "deadline_uplift": state["deadline_uplift"][row["profession"]],
        "monthly": monthly,
        "cluster": {**cluster, "profile": {k: round(v, 3) for k, v in profile.items()}},
    })
```

10.4 Frontend call to the backend (static/app.js)

```
async function onPredict(e) {
  e.preventDefault();
  const btn = $("#predict-form button");
  btn.disabled = true;
  try {
    renderResults(await api("/api/predict", { method: "POST", body: readForm() }));
  } catch (err) {
    toast(err.message, true);
  } finally {
    btn.disabled = false;
  }
}
```

11. Training Data

Sample of 18 rows (every 500th row) from data/token_usage_daily.csv, which has 9,000 rows in total. The last three columns are the values the model learns to predict.

User	Profession	Exp (yrs)	Model tier	Agents	Day	Deadline wk	Sessions	Msgs / session	Prompt words	Docs	Code task	Input tokens	Output tokens	Total tokens
U001	Software Developer	3	medium	0	Sat	0	4	6	268	0	1	60,996	10,805	71,801
U017	Digital Marketer	1	medium	0	Wed	1	8	4	104	0	0	22,884	7,090	29,974
U034	Designer	3	large	0	Thu	0	5	3	113	1	0	9,141	3,296	12,437
U051	Student	3	large	0	Wed	0	12	5	109	2	0	60,842	14,904	75,746
U067	Doctor	3	medium	0	Tue	0	2	4	76	1	0	6,351	1,216	7,567
U084	Business Analyst	9	large	0	Sun	0	5	8	161	0	0	69,666	11,169	80,835
U101	Digital Marketer	22	large	0	Mon	0	5	3	111	3	0	15,835	6,923	22,758
U117	Researcher	18	medium	0	Sun	0	2	2	392	4	0	14,000	2,512	16,512
U134	Data Scientist	12	medium	0	Mon	0	6	4	168	4	0	33,682	6,978	40,660
U151	Doctor	20	medium	0	Mon	0	0	4	83	1	0	2,518	0	2,518
U167	Customer Support	17	small	1	Sun	0	1	8	67	0	0	10,885	1,970	12,855
U184	Content Writer	15	large	0	Fri	1	9	7	147	2	0	1,07,810	21,385	1,29,195
U201	Researcher	4	medium	0	Sat	1	1	6	383	1	0	23,874	2,059	25,933
U217	Software Developer	11	small	0	Thu	0	7	7	258	1	1	1,24,812	28,150	1,52,962
U234	Teacher	9	medium	0	Wed	0	6	5	132	1	0	36,378	11,920	48,298
U251	Customer Support	5	small	0	Sat	0	2	5	58	0	0	4,753	1,631	6,384
U267	Student	1	large	0	Tue	0	3	5	129	1	0	21,318	4,208	25,526
U284	Lawyer	6	medium	0	Tue	1	4	6	315	4	0	71,945	9,040	80,985

Average daily usage per profession in the full training data:

Profession	Rows	Avg input tokens	Avg output tokens	Avg total tokens
Software Developer	750	1,40,669	21,590	1,62,259
Data Scientist	750	1,37,447	17,210	1,54,657
Content Writer	750	1,07,555	26,087	1,33,642
Researcher	750	1,06,604	13,914	1,20,518
Customer Support	750	66,615	14,334	80,949

Profession	Rows	Avg input tokens	Avg output tokens	Avg total tokens
Business Analyst	750	65,972	10,729	76,701
Digital Marketer	750	62,141	14,160	76,301
Lawyer	750	54,849	8,119	62,968
Student	750	50,630	8,769	59,399
Teacher	750	40,978	9,010	49,988
Designer	750	17,742	4,137	21,878
Doctor	750	15,850	3,322	19,172